

CHECKLIST

AI-Prompt – Claude Sonnet 4.6

This is a high-level, structured prompt designed to be given to a Senior Security Engineer or an AI Architect. It frames your current progress as a successful "Phase 1" and demands a rigorous, multi-layered validation and stress-test "Phase 2."

The Prompt

Subject: Structural Validation and Cyber-Resiliency Stress Test for AWS-Elastic SOC Architecture

Context:

The "Foundational Phase" of our Cloud SOC is complete. We have successfully established a dual-VPC architecture (Main VPC and Security VPC) with a peering bridge.

Current Deployment State:

- 1. **SOC Platform:** Elastic Fleet Server is operational within the Security VPC.
- 2. **Endpoint Coverage:** Elastic Agents are successfully deployed on Frontend and Backend EC2 instances in the Main VPC.
- 3. **Telemetry Ingestion:** The following data streams are integrated and ostensibly active:
 - o **Governance:** CloudTrail via S3 (aws-cloudtrail-logs-mj).
 - o **Network:** VPC Flow Logs via CloudWatch.
 - o **Storage Audit:** S3 Access Logs (company-storage-network-project).
 - o **Database:** RDS MySQL Error/General logs and Performance Metrics.
 - o **Infrastructure:** Global EC2 Metrics (SOC, Frontend, and Backend).

The Objective:

Transition from "Deployment" to "Validation." I require an exhaustive, futuristic checklist and testing framework to verify that this SOC is not just receiving data, but is battle-ready.

1. High-Level Testing Strategy

Please provide a table summarizing the testing categories required to validate this architecture.

Category	Purpose	Strategic Objective

Network Integrity	Validating the cross-VPC peering and private routing.	Ensure zero-latency, secure communication between Main and Security VPCs.
Telemetry Fidelity	Verifying the accuracy and parsing of incoming logs.	Ensure data is actionable and mapped correctly to the Elastic Common Schema (ECS).
Cloud Service Sync	Testing the "Pull" mechanism from AWS native services.	Guarantee that RDS, S3, and CloudTrail streams are real-time and persistent.
Operational Health	Monitoring the monitoring system.	Ensure the SOC instance and Agents are resilient to high resource load.
Threat Simulation	Triggering alerts via simulated attacks.	Validate the SIEM's ability to detect and visualize adversarial behavior.

2. The Granular Checklist

For each category, provide a detailed breakdown of specific testing elements. Each element must include:

- **Description:** What is the specific test?
- **Purpose:** Why are we testing this?
- **Objective:** What is the "Success Condition"?

Category A: Network & Communication Hub

- **Element A.1: Cross-VPC Peering Latency**
- **Element A.2: Port 9200/8220 Isolation** (Ensuring only the VPC-CIDR can reach the SOC).
- **Element A.3: Agent Heartbeat Continuity.**

Category B: Log Ingestion & Parsing (Fidelity)

- **Element B.1: EC2 Auth Log Correlation** (Testing SSH login attempts).

- **Element B.2: CloudTrail API Mapping** (Verifying IAM changes show up in Kibana).
- **Element B.3: RDS Slow Query Identification.**

Category C: Cloud Native "Pull" Validation

- **Element C.1: S3 Bucket Enumeration Alert** (Testing the S3 Access logs).
- **Element C.2: VPC Flow Log "REJECT" Visibility.**
- **Element C.3: Metric Aggregation Accuracy.**

Category D: Adversarial Simulation (Security Testing)

- **Element D.1: Layer 7 DDoS Simulation** (Testing the Frontend's stress and Kibana's visibility).
- **Element D.2: Malicious Payload Upload** (Testing S3/Frontend validation).
- **Element D.3: SQL Injection (SQLi) Probing** (Targeting RDS through the Frontend).

Strategies

 High-Level Testing Strategy

Category	Purpose	Strategic Objective
A: Network & Communication	Validate cross-VPC peering and private routing	Ensure secure, isolated communication between Main and Security VPCs
B: Log Ingestion & Parsing	Verify accuracy and parsing of incoming logs	Ensure data is actionable and mapped to ECS schema
C: Cloud Native Pull Validation	Test the pull mechanism from AWS services	Guarantee CloudTrail, S3, RDS streams are persistent and real-time
D: Adversarial Simulation	Trigger alerts via simulated attacks	Validate SIEM's ability to detect and visualize adversarial behavior

AI (Claude Sonnet 4.6 & Gemini 3), Documentations, YouTube, and self-brainstorming were performed.

Category A: Network & Communication Hub

A.1: Cross-VPC Peering Latency

Description: Verify that the SOC instance in Security VPC can communicate with Frontend and Backend in Main VPC with acceptable latency and zero packet loss.

Purpose: VPC peering can be established but silently broken by route table misconfigurations. We need to confirm it's stable and fast.

Success Condition: Less than 2ms latency between VPCs, zero packet loss over 100 pings.

How to Test:

From your **SOC instance** via SSM:

Ping Frontend private IP

```
ping -c 100 <FRONTEND_PRIVATE_IP>
```

Ping Backend private IP

```
ping -c 100 <BACKEND_PRIVATE_IP>
```

From your **Frontend EC2** via SSM:

Ping SOC private IP

```
ping -c 100 10.1.1.25
```

Expected output:

```
100 packets transmitted, 100 received, 0% packet loss
```

```
rtt min/avg/max = 0.3/0.5/1.2 ms
```

If ping is blocked (ICMP might be blocked by Security Groups), use this instead:

Test TCP connectivity on port 9200 from both frontend and backend instances

```
curl -s -o /dev/null -w "Response time: %{time_connect}s\n" http://10.1.1.25:9200
```

From SSM SOC Platform in the Security VPC:

```
curl -s -o /dev/null -w "Response time: %{time_connect}s\n"
http://<FRONTEND_PRIVATE_IP>:80
```

```
curl -s -o /dev/null -w "Response time: %{time_connect}s\n"
http://<BACKEND_PRIVATE_IP>:80
```

```
$ curl -s -o /dev/null -w "Response time: %{time_connect}s\n" http://10.0.1.252:80
Response time: 0.000749s
$ curl -s -o /dev/null -w "Response time: %{time_connect}s\n" http://10.0.2.37:80
Response time: 0.000000s
$
```

The backend curl does not work because the port 80 of that instance has the frontend SG. But it's okay.

✓ Check in Kibana:

Go to **Fleet** → **Agents** — all three agents (SOC, Frontend, Backend) should show **Last Activity** within the last 30 seconds. If peering was broken, the Main VPC agents would show as offline.

☐	Status	Host ↕	Agent policy ↕	CPU ⓘ	Memory ⓘ	Last activity ↕	Version ↕
☐	Healthy	ip-10-0-1-252	Main-VPC-EC2-Policy rev. 13	1.13 %	189 MB	16 seconds ago	9.3.0+build202602051825
☐	Healthy	ip-10-0-2-37	Main-VPC-EC2-Policy rev. 13	1.22 %	165 MB	21 seconds ago	9.3.0+build202602051825
☐	Healthy	ip-10-1-1-25	Fleet Server Policy rev. 12	1.63 %	396 MB	7 seconds ago	9.3.0

Rows per page: 20 ▾

A.2: Port 9200/8220 Isolation

Description: Verify that Elasticsearch (9200) and Fleet Server (8220) are ONLY reachable from within the VPC CIDR range — not from the public internet.

Purpose: If these ports are accidentally exposed publicly, anyone could query your Elasticsearch or enroll rogue agents into your Fleet. This is a critical security check.

Success Condition: Ports 9200 and 8220 return connection refused or timeout from any IP outside your VPC CIDRs.

How to Test:

From your local laptop (outside AWS):

This should FAIL/timeout - good!

```
curl http://<SOC_PUBLIC_IP>:9200
```

```
curl http://<SOC_PUBLIC_IP>:8220
```

Your SOC is in a private subnet so it has no public IP — this should already be impossible. But verify your Security Group rules:

Go to **AWS Console** → **EC2** → **SOC Instance** → **Security Groups** → **Inbound Rules**

You should see:

- Port **9200** — Source: **Main VPC CIDR only** (e.g. **10.0.0.0/16**) ✓
- Port **8220** — Source: **Main VPC CIDR only** ✓
- Port **5601** — Source: **Security VPC CIDR only** (e.g. **10.1.0.0/16**) ✓
- **NO rules with source 0.0.0.0/0 on any of these ports** ✓

If you see **0.0.0.0/0** on any of these — **delete from the inbound rules immediately**.

Verify from Main VPC (should work):

From Frontend EC2 - this SHOULD succeed

```
curl -k https://10.1.1.25:8220
```

Expected: "Client sent HTTP request to HTTPS server" = port is reachable ✓

From Frontend EC2 - this SHOULD succeed

```
curl http://10.1.1.25:9200
```

Expected: Elasticsearch JSON response ✓

If you mean with this, it's also fine. It means both are reachable, but no page was found and incorrect authentication.

```
ubuntu@ip-10-0-1-252:~$ curl -k https://10.1.1.25:8220
404 page not found
ubuntu@ip-10-0-1-252:~$ curl http://10.1.1.25:9200
{"error":{"root_cause":{"type":"security_exception","reason":"missing authentication credentials for REST request [/]","header":{"WWW-Authenticate":["Basic realm=\"security\", charset=\"UTF-8\"", "ApiKey"]}}},"type":"security_exception","reason":"missing authentication credentials for REST request [/]","header":{"WWW-Authenticate":["Basic realm=\"security\", charset=\"UTF-8\"", "ApiKey"]}}, "status":401}ubuntu@ip-10-0-1-252:~$
```

A.3: Agent Heartbeat Continuity

Description: Stress test the agent connection by simulating a brief network interruption and verifying agents automatically reconnect.

Purpose: In a real SOC, network blips happen. Agents must self-heal and resume log shipping without manual intervention.

Success Condition: After a 60-second network interruption, agents reconnect automatically and resume sending logs within 2 minutes.

How to Test:

On your **Frontend EC2** via SSM:

Stop the agent temporarily

```
sudo systemctl stop elastic-agent
```

Wait 60 seconds

```
sleep 60
```

Start it again

```
sudo systemctl start elastic-agent
```

Check recovery

```
sudo elastic-agent status
```

Also test that the agent survives a reboot:

Verify agent is set to start on boot

```
sudo systemctl is-enabled elastic-agent
```

Expected: "enabled"

Reboot the instance

```
sudo reboot
```

After reboot, SSH back in and check:

```
sudo elastic-agent status
```

Expected: HEALTHY within 2 minutes of boot

✓ Check in Kibana:

1. Go to Fleet → Agents
2. Watch the Frontend agent status during the stop — it will go Offline
3. After restart it should return to Healthy
4. Go to Discover and search:

`agent.name: "ip-10-0-1-252" AND event.dataset: "system.syslog"`

You should see a gap in logs during the stop period, then logs resume — this confirms the agent recovered cleanly.

I hereby confirmed that this agent survives during network interruptions ✓

Category A Summary Checklist

Element	Test	Success Condition
A.1 Peering Latency	ping -c 100 across VPCs	0% packet loss, <2ms
A.2 Port Isolation	Check Security Group rules, test from outside	No public exposure on 9200/8220
A.3 Heartbeat Continuity	Stop/start agent, reboot test	Auto-reconnects, enabled on boot

Category B: Log Ingestion & Parsing (Fidelity)

This is where it gets exciting — we're going to **trigger real security events** and watch them appear in Kibana live.

B.1: EC2 Auth Log Correlation

Description: Simulate failed SSH login attempts and verify they appear in Kibana with correct parsing.

Purpose: Auth logs are your first line of defence for detecting brute force attacks and unauthorized access attempts.

Success Condition: Failed login attempts appear in Kibana within 60 seconds, correctly identified with source IP, username attempted, and timestamp.

How to Test:

From your **local laptop**, attempt a fake SSH login to the Frontend public IP:

Run this 5 times from PowerShell - use the wrong password intentionally

```
ssh fakeuser@<FRONTEND_PUBLIC_IP>
```

Type a wrong password each time and let it fail. This generates **authentication failure** entries in **/var/log/auth.log**.

Then verify on the **Frontend EC2** via SSM, Tabby, or PuTTY:

Confirm the failed attempts are in the auth log

```
sudo tail -20 /var/log/auth.log
```

You should see lines like:

```
Failed password for invalid user fakeuser from <YOUR_IP> port 12345 ssh2
```

 Check in Kibana:

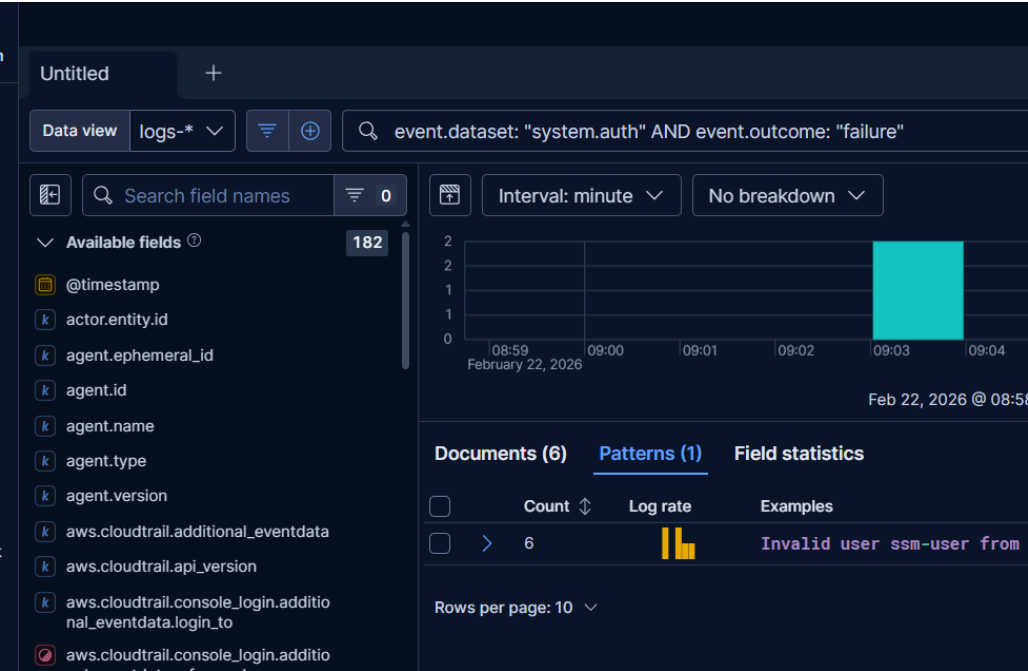
1. Go to Kibana → Discover
2. In the index pattern dropdown, select `logs-system\*`
3. In the search bar, type:

```
event.dataset: "system.auth" AND event.outcome: "failure"
```

4. Set the time range to **Last 15 minutes**
5. You should see the failed login events appear with fields like:
 - `source.ip` — your laptop's IP
 - `user.name` — fakeuser
 - `event.outcome` — failure
 - `host.name` — the Frontend EC2

I used both my local laptop and frontend machine to SSH into the frontend machine in the main VPC. Here are the results:

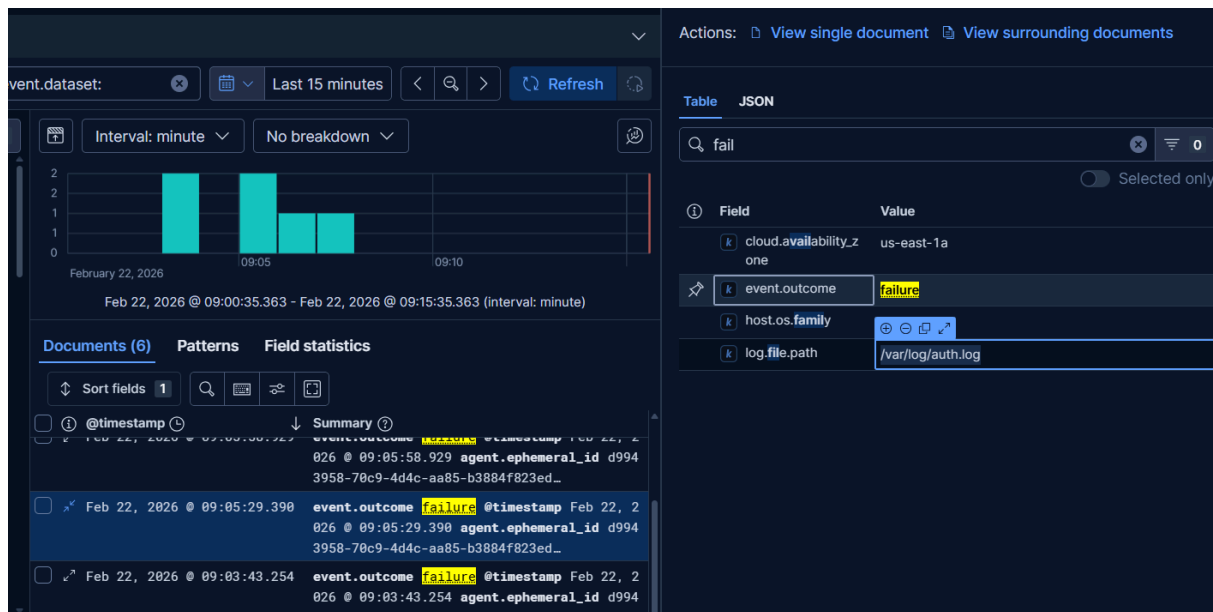
```
ubuntu@ip-10-0-1-252:/var/log$ sudo tail -20 /var/log/auth.log
2026-02-22T01:55:01.060952+00:00 ip-10-0-1-252 CRON[8568]: pam_unix(cron:session): ses
2026-02-22T02:01:48.483617+00:00 ip-10-0-1-252 sshd[8660]: Accepted publickey for ubun
2026-02-22T02:01:48.485400+00:00 ip-10-0-1-252 sshd[8660]: pam_unix(sshd:session): ses
2026-02-22T02:01:48.522819+00:00 ip-10-0-1-252 systemd-logind[569]: New session 66 of
2026-02-22T02:01:48.544278+00:00 ip-10-0-1-252 (systemd): pam_unix(systemd-user:session)
2026-02-22T02:03:36.815733+00:00 ip-10-0-1-252 sshd[8834]: Invalid user ssm-user from
2026-02-22T02:03:37.093225+00:00 ip-10-0-1-252 sshd[8834]: Connection reset by invalid
2026-02-22T02:03:43.254187+00:00 ip-10-0-1-252 sshd[8840]: Invalid user ssm-user from
2026-02-22T02:03:43.577950+00:00 ip-10-0-1-252 sshd[8840]: Connection reset by invalid
2026-02-22T02:05:01.067278+00:00 ip-10-0-1-252 CRON[8858]: pam_unix(cron:session): ses
2026-02-22T02:05:01.070300+00:00 ip-10-0-1-252 CRON[8858]: pam_unix(cron:session): ses
2026-02-22T02:05:29.390290+00:00 ip-10-0-1-252 sshd[8866]: Invalid user ssm-user from
2026-02-22T02:05:29.740997+00:00 ip-10-0-1-252 sshd[8866]: Connection reset by invalid
2026-02-22T02:05:58.929228+00:00 ip-10-0-1-252 sshd[8875]: Invalid user ssm-user from
2026-02-22T02:05:59.241213+00:00 ip-10-0-1-252 sshd[8875]: Connection reset by invalid
2026-02-22T02:06:55.326165+00:00 ip-10-0-1-252 sshd[8887]: Invalid user ssm-user from
2026-02-22T02:06:55.657859+00:00 ip-10-0-1-252 sshd[8887]: Connection reset by invalid
2026-02-22T02:07:34.822750+00:00 ip-10-0-1-252 sshd[8898]: Invalid user ssm-user from
2026-02-22T02:07:35.091487+00:00 ip-10-0-1-252 sshd[8898]: Connection reset by invalid
2026-02-22T02:09:21.592370+00:00 ip-10-0-1-252 sudo:  ubuntu : TTY=pts/3 ; PWD=/var/l
ubuntu@ip-10-0-1-252:/var/log$
```



Discover / .ds-logs-system.auth-default-2026.02.21-000001#AZyDGt-fl292pN9OxnIq

Table JSON

```
1 {
2   "_index": ".ds-logs-system.auth-default-2026.02.21-000001",
3   "_id": "AZyDGt-fl292pN9OxnIq",
4   "_version": 1,
5   "_source": {
6     "@timestamp": "2026-02-22T02:07:34.822Z",
7     "agent": {
8       "ephemeral_id": "d9943958-70c9-4d4c-aa85-b3884f823ed8",
9       "id": "5061dc19-f9d6-4585-9aed-cbdb3b5ae439",
10      "name": "ip-10-0-1-252",
11      "type": "filebeat",
12      "version": "9.3.0+build202602051825"
13    },
14    "cloud": {
15      "account": {
16        "id": "967883358429"
17      },
18      "availability_zone": "us-east-1a",
19      "image": {
20        "id": "ami-0b6c6ebed2801a5cb"
21      },
22      "instance": {
23        "id": "i-03035f16a3cd073d6"
24      },
25      "machine": {
26        "type": "t2.micro"
```



You have to confirm that the `log.file.path` is in the right spot, as shown in this picture.

B.2: CloudTrail API Mapping

Description: Make a deliberate IAM change in AWS Console and verify it appears in Kibana via CloudTrail.

Purpose: CloudTrail is your governance layer — every AWS API call should be visible. If an attacker creates a backdoor IAM user, you'll see it here.

Success Condition: The IAM event appears in Kibana within 5 minutes, with the correct `event.action`, `user.name`, and `aws.cloudtrail.user_identity` fields.

How to Test:

Go to AWS Console → IAM → Users and do something simple and harmless:

1. Click any existing user
2. Click the "Tags" tab
3. Click "Add tag"
4. Key: `soc-test` Value: `validation`
5. Click "Save"

This generates a `TagUser` CloudTrail event. After 2-3 minutes, delete the tag to keep things clean.

✓ Check in Kibana:

1. Go to Discover
2. Select index `logs-aws.cloudtrail-\*`
3. Search:

```
event.dataset: "aws.cloudtrail" AND event.action: "TagUser"
```

4. Set time range to **Last 15 minutes**
5. Click the event and expand it — look for:
 - `aws.cloudtrail.user_identity.arn` — who made the change
 - `event.action` — TagUser
 - `cloud.region` — us-east-1
 - `@timestamp` — when it happened

I created a new IAM user with admin privileges with a tag (key=SOC, value=Platform-Real-Son).

Search for the user deletion:

```
event.dataset: "aws.cloudtrail" AND event.action: "DeleteUser"
```

Search for the tag deletion:

```
event.dataset: "aws.cloudtrail" AND event.action: "UntagUser"
```

Or search for everything related to your test user at once:

```
event.dataset: "aws.cloudtrail" AND aws.cloudtrail.request_parameters: *<testusername>*
```

Replace `<testusername>` with whatever you named the test IAM user.

Click on any result and expand it. You should see:

- `event.action` — DeleteUser or UntagUser

- `aws.cloudtrail.user_identity.arn` — your admin account that performed the deletion
- `@timestamp` — when it happened
- `aws.cloudtrail.request_parameters` — the username that was deleted

(Add a tag to the user)

The screenshot shows the AWS CloudTrail console interface. On the left, there's a sidebar with 'Available fields' and a search bar. The main area displays a log event for 'Feb 22, 2026 @ 09:26:31.000'. The event details show the user 'MySon' with tags 'SOC' and 'Platform-Real-Son'. The event type is 'DeleteUser' and the event dataset is 'aws.cloudtrail'.

Field	Value
aws.cloudtrail.event.category	Management
aws.cloudtrail.event.type	AwsApiCall
aws.cloudtrail.event.version	1.11
aws.cloudtrail.flattened.request.parameters	{ "userName": "MySon", "tags": { "key": "SOC", "value": "Platform-Real-Son" } }
aws.cloudtrail.management.event	true
aws.cloudtrail.read_only	false
aws.cloudtrail.recipient_account_id	967883358429

(Delete user)

The screenshot shows the AWS CloudTrail console interface. On the left, there's a sidebar with 'Available fields' and a search bar. The main area displays a log event for 'Feb 22, 2026 @ 09:30:54.000'. The event details show the user 'MySon' with tags 'SOC' and 'Platform-Real-Son'. The event type is 'DeleteUser' and the event dataset is 'aws.cloudtrail'.

Field	Value
event.type	[user deletion]
input.type	aws-s3
log.file.path	https://aws-cloudtrail-logs-mj.s3.us-east-1.amazonaws.com/AWSLogs/967883358429/CloudTrail/us-east-1/2026/02/22/967883358429_CloudTrail_us-east-1_20260222T0235Z_nngQSEYoL3VGjftO.json.gz
log.offset	74,527
related.entity	[ASIA6CWSYADOVSSLKUYP, MySon, arn:aws:iam::967883358429:root]
related.user	[MySon, 967883358429]
source.address	58.97.216.147
source.as.number	131,178
source.as.organiza	EZECOM CO., LTD.

B.3: RDS Slow Query Identification

Description: Trigger a deliberately slow MySQL query and verify it appears in the RDS logs flowing into Kibana.

Purpose: Slow queries can indicate SQL injection attempts, misconfigured application code, or database abuse.

Success Condition: The slow query appears in Kibana in the CloudWatch log stream within 5 minutes.

How to Test:

You need to connect to your RDS instance. From your **Backend EC2** via SSM:

Install MySQL client if not already there

```
sudo apt-get install mysql-client -y
```

Connect to RDS

```
mysql -h <RDS_ENDPOINT> -u <DB_USERNAME> -p
```

Once connected, run a deliberately slow query:

```
SELECT SLEEP(5);
```

```
SELECT SLEEP(5);
```

```
SELECT SLEEP(5);
```

This forces MySQL to sleep for 5 seconds — anything over your slow query threshold gets logged. Type **exit** when done.

Also trigger some failed login attempts to RDS:

```
mysql -h <RDS_ENDPOINT> -u fakeuser -pwrongpassword
```

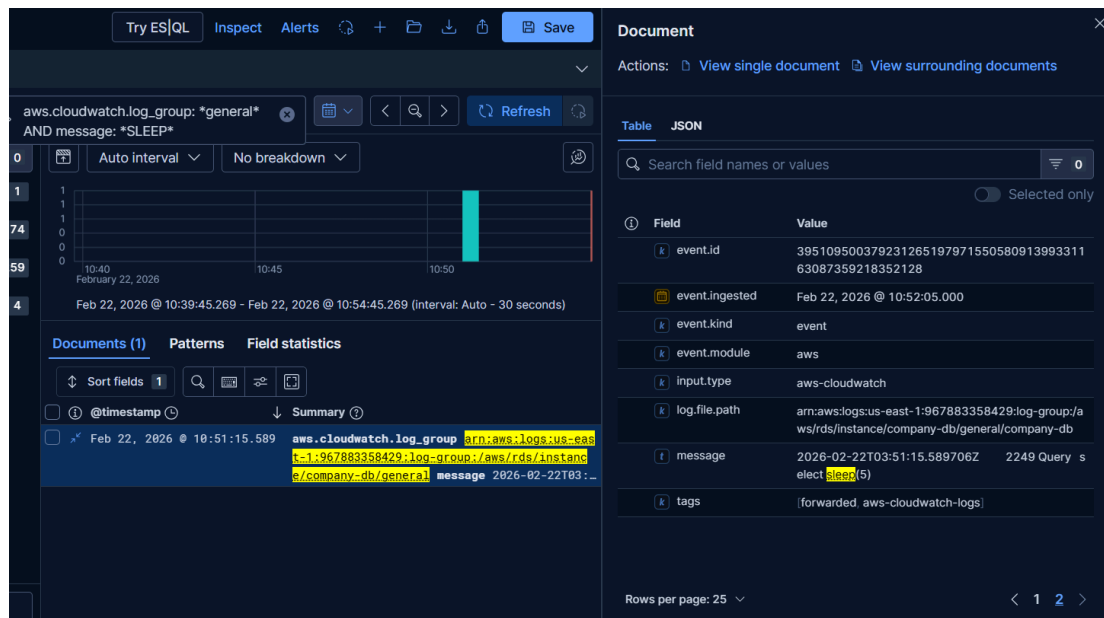
✅ Check in Kibana:

1. Go to Discover
2. Select index `logs-aws.cloudwatch-\*`

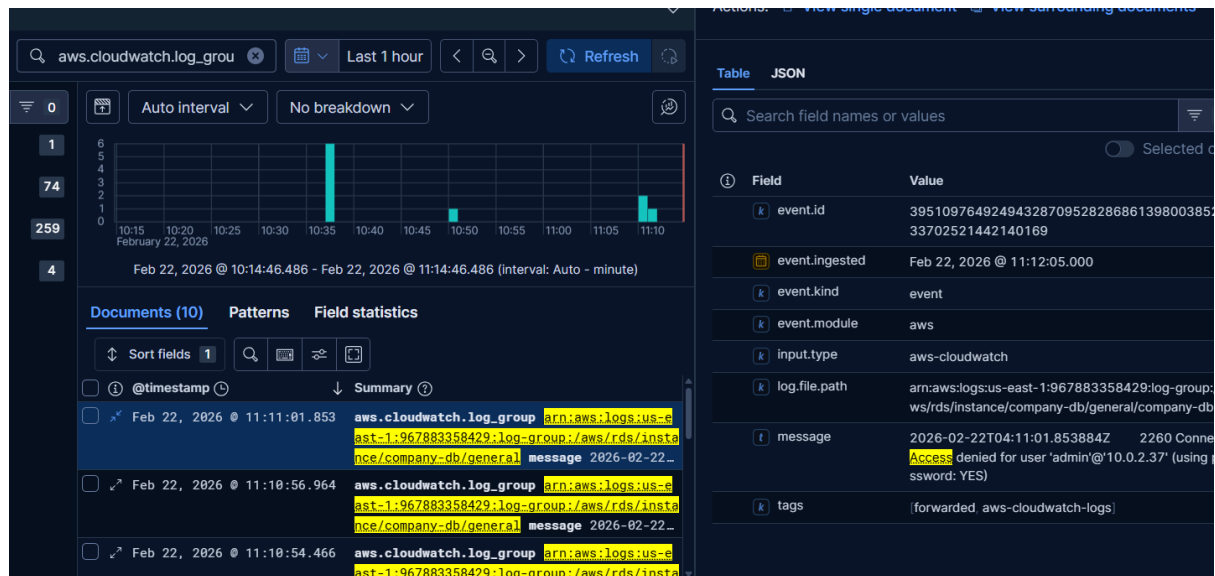
3. Search:

aws.cloudwatch.log_group: *general* AND message: *SLEEP*

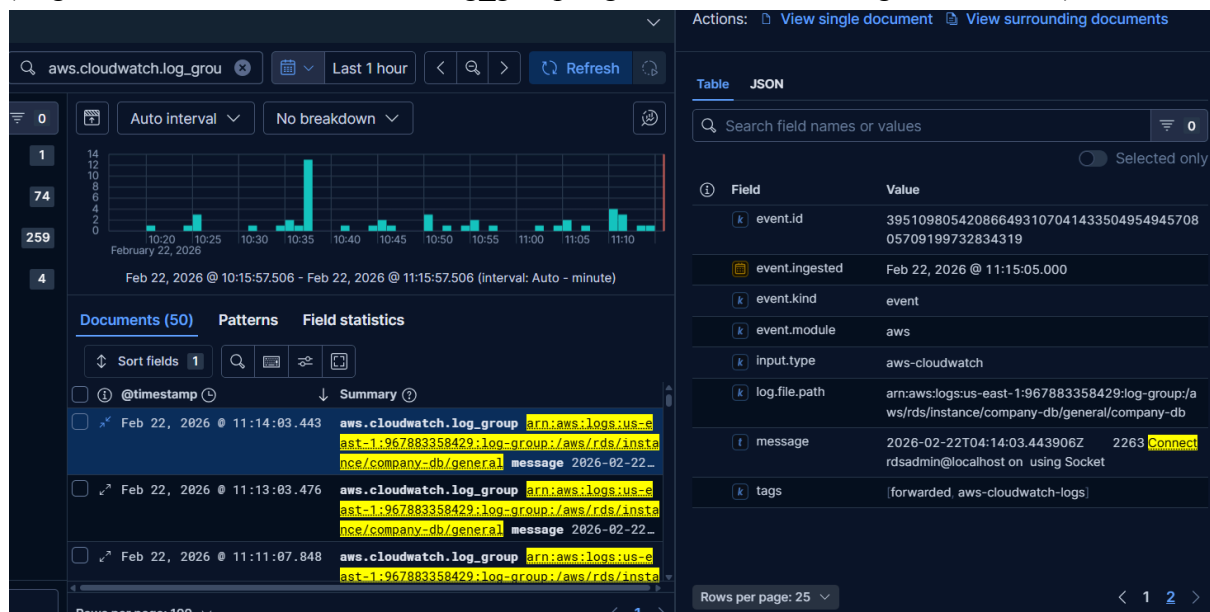
4. Set time range to **Last 15 minutes**
5. Look for SLEEP query entries and failed authentication entries



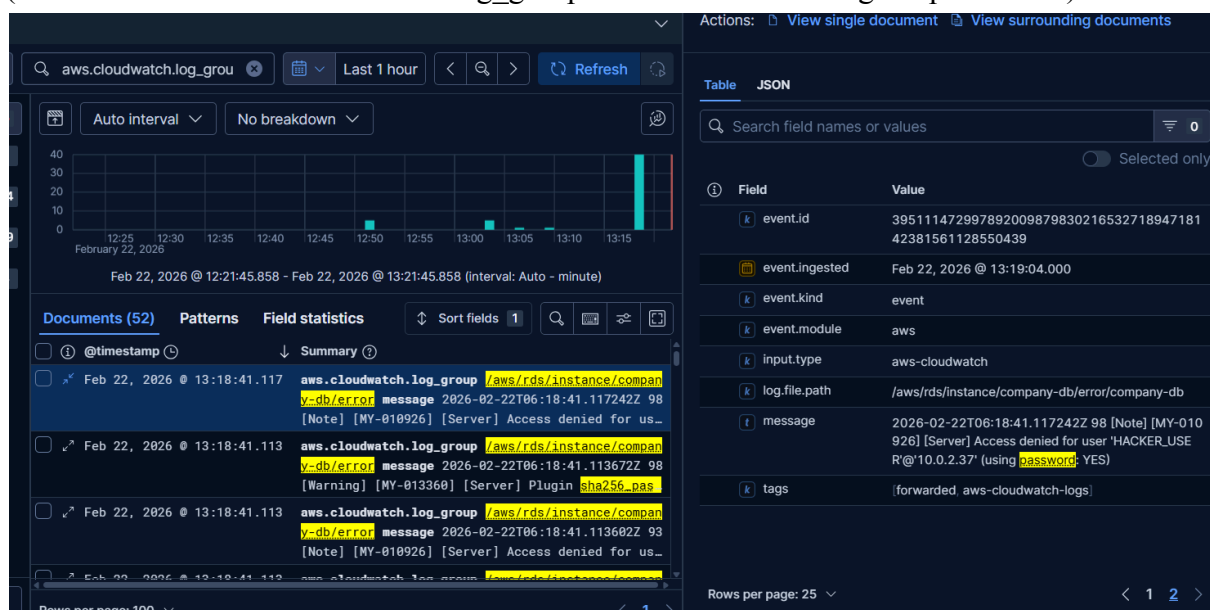
(Access Denied – aws.cloudwatch.log_group: *general* AND message : *access*)



(Login Success– aws.cloudwatch.log_group: *general* AND message : *connect*)



(Access Denied – aws.cloudwatch.log_group: *error* AND message : *password*)



Note:

If you are facing errors that the /error logs from CloudWatch in AWS did not show any logs, you need to do more. Attack more.

You can try this attack when accessing MySQL:

```
#!/bin/bash
```

```
for i in {1..20}; do
```

```
mysql -h <DATABASE-ENDPOINT> -u HACKER_USER -pWRONG_PASSWORD -e  
"exit" &
```

```
done
```

What you're experiencing is a classic security engineering scenario called Verbosity Filtering (or "The Selective Bouncer" effect).

Why this happens (The short version)

By default, MySQL 8.0 treats a failed password as a "Note"—a minor piece of information—rather than a critical "Error".

- The Default: MySQL is set to `log_error_verbosity = 2`, which means it only writes actual crashes and serious warnings to the log. It ignores "Notes" to keep the log file small and clean.
- The Reboot: When you restart the database, the engine writes "System Starting" logs. These are considered mandatory system events, so they bypass the filter and show up immediately.
- The Attack: When you run your password attack, the bouncer (MySQL) sees it, but because your settings say "only tell me about serious problems," he doesn't write it down.

What is this scenario called?

This is known as a Low-Fidelity Logging State. In a SOC, this is dangerous because you are "blind" to brute-force attacks. By changing the verbosity to 3, you switched to High-Fidelity Logging, where the bouncer is forced to write down every single suspicious interaction, no matter how small.

Category B Summary Checklist

Element	Test	Success Condition
B.1 Auth Log Correlation	Failed SSH logins	Appears in <code>logs-system*</code> within 60 seconds
B.2 CloudTrail API Mapping	IAM tag change	Appears in <code>logs-aws.cloudtrail-*</code> within 5 minutes
B.3 RDS Slow Query	SLEEP(5) query	Appears in <code>logs-aws.cloudwatch-*</code> within 5 minutes

Category C: Cloud Native Pull Validation

C.1: S3 Bucket Enumeration Alert

Description: Simulate an attacker listing the contents of your S3 bucket repeatedly and verify it shows up in your S3 access logs in Kibana.

Purpose: Bucket enumeration is one of the first things attackers do after gaining AWS access. Your SOC should catch this.

Success Condition: Multiple `ListBucket` events appear in Kibana within 15 minutes.

Before Running — Confirm S3 Access Logging is Enabled:

Go to **AWS Console** → **S3** → **company-storage-network-project** or **your main company production storage** → **Properties tab** → **Server access logging**

Confirm it shows:

- **Status:** Enabled 

- **Target bucket:** company-storage-access-logs  (Please refer to this documentation: Add SOC/AWS Integration/S3 Access Log)

If not enabled, turn it on first before running the test.

How to Test:

From your **SOC instance** via SSM:

Type the following CLIs:

```
cd ~
```

```
mkdir home
```

```
cd home/
```

```
bash
```

```
sudo apt update && sudo apt install unzip curl -y
```

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
```

```
unzip awscliv2.zip
```

```
sudo ./aws/install
```

```
aws --version
```

```
nano script.sh
```

Then paste:

```
for i in {1..10}; do
```

```
aws s3 ls s3://company-storage-network-project --region us-east-1
```

```
sleep 2
```

```
done
```

Save & exit.

```
chmod +x script.sh
```

Then, type: `./script.sh`

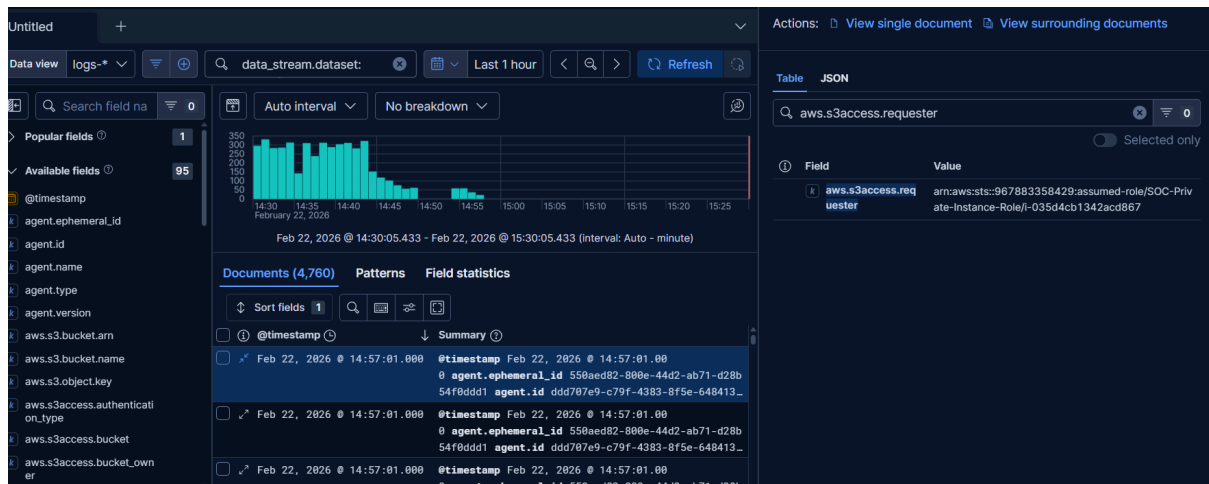
✓ Check in Kibana:

Wait 15 minutes first — S3 access logs have a longer delay than CloudWatch. Then search for the last 30 minutes: `data_stream.dataset: "aws.s3access"`

Look for:

- `aws.s3access.operation` — `ListBucket`
- `aws.s3access.http_status` — `200`
- `aws.s3access.requester` — your SOC role ARN

After about 30 minutes:



C.2: VPC Flow Log REJECT Visibility

Description: Generate traffic that gets blocked by a Security Group and verify the REJECT entries appear in Kibana via VPC Flow Logs.

Purpose: REJECT entries in Flow Logs reveal port scanning, unauthorized access attempts, and misconfigured applications trying to reach blocked ports.

Success Condition: REJECT flow entries appear in Kibana showing source IP, destination port, and REJECT action.

How to Test:

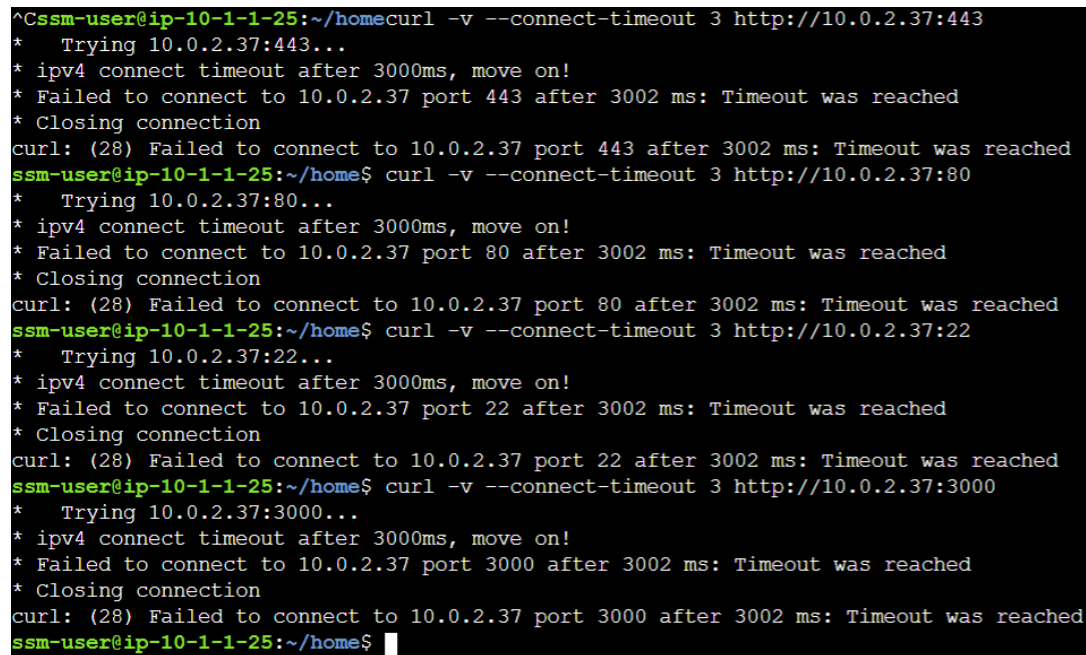
From your SOC instance via SSM, try connecting to a port that's definitely blocked on your Backend EC2:

Try blocked ports on Backend EC2

```
curl -v --connect-timeout 3 http://<BACKEND_PRIVATE_IP>:22
```

```
curl -v --connect-timeout 3 http://<BACKEND_PRIVATE_IP>:3306
```

```
curl -v --connect-timeout 3 http://<BACKEND_PRIVATE_IP>:443
```



```
^C ssm-user@ip-10-1-1-25:~/home$ curl -v --connect-timeout 3 http://10.0.2.37:443
* Trying 10.0.2.37:443...
* ipv4 connect timeout after 3000ms, move on!
* Failed to connect to 10.0.2.37 port 443 after 3002 ms: Timeout was reached
* Closing connection
curl: (28) Failed to connect to 10.0.2.37 port 443 after 3002 ms: Timeout was reached
ssm-user@ip-10-1-1-25:~/home$ curl -v --connect-timeout 3 http://10.0.2.37:80
* Trying 10.0.2.37:80...
* ipv4 connect timeout after 3000ms, move on!
* Failed to connect to 10.0.2.37 port 80 after 3002 ms: Timeout was reached
* Closing connection
curl: (28) Failed to connect to 10.0.2.37 port 80 after 3002 ms: Timeout was reached
ssm-user@ip-10-1-1-25:~/home$ curl -v --connect-timeout 3 http://10.0.2.37:22
* Trying 10.0.2.37:22...
* ipv4 connect timeout after 3000ms, move on!
* Failed to connect to 10.0.2.37 port 22 after 3002 ms: Timeout was reached
* Closing connection
curl: (28) Failed to connect to 10.0.2.37 port 22 after 3002 ms: Timeout was reached
ssm-user@ip-10-1-1-25:~/home$ curl -v --connect-timeout 3 http://10.0.2.37:3000
* Trying 10.0.2.37:3000...
* ipv4 connect timeout after 3000ms, move on!
* Failed to connect to 10.0.2.37 port 3000 after 3002 ms: Timeout was reached
* Closing connection
curl: (28) Failed to connect to 10.0.2.37 port 3000 after 3002 ms: Timeout was reached
ssm-user@ip-10-1-1-25:~/home$
```

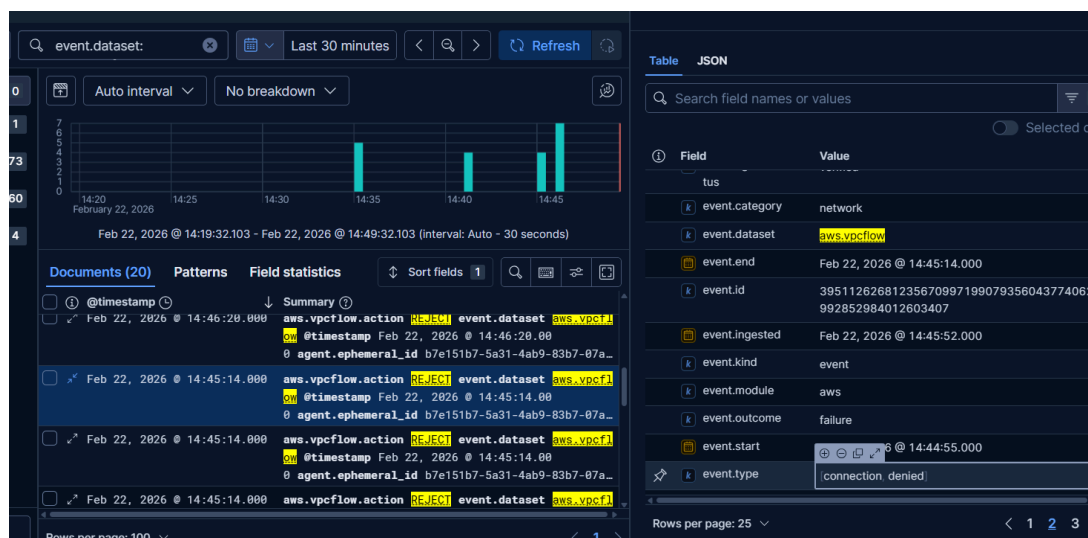
These should all time out — meaning the Security Group is rejecting them and VPC Flow Logs are recording the REJECTs.

✓ Check in Kibana:

```
event.dataset: "aws.vpcflow" AND aws.vpcflow.action: "REJECT"
```

Look for fields:

- aws.vpcflow.action — REJECT
- destination.port — 22, 3306, 443
- source.ip — your SOC private IP 10.1.1.25
- destination.ip — your Backend private IP



C.3: Metric Aggregation Accuracy

Description: Stress your EC2 instances and verify the CPU/Memory spike shows up correctly in Kibana metrics.

Purpose: If metrics aren't accurate, your SOC is blind to performance-based attacks like cryptomining, DDoS bots, or resource exhaustion.

Success Condition: CPU spike of 80%+ appears in Kibana within 2 minutes of being triggered.

How to Test:

On your Frontend EC2 via SSM:

Install stress tool

```
sudo apt-get install stress -y
```

Spike CPU to 100% for 60 seconds

```
sudo stress --cpu 4 --timeout 60
```

On your Backend EC2 via SSM simultaneously:

```
sudo apt-get install stress -y
```

```
sudo stress --cpu 4 --timeout 60
```

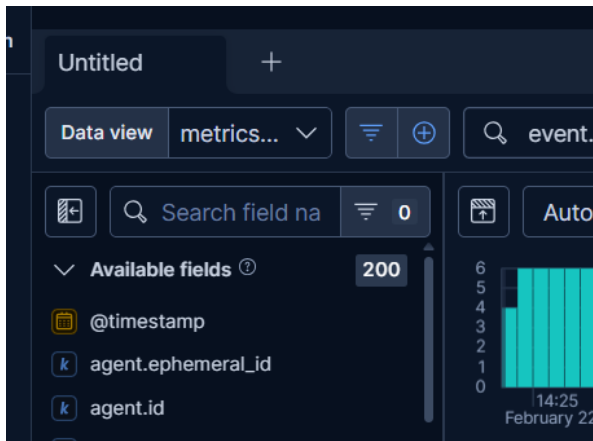
✅ **Check in Kibana:**

1. Go to Kibana → Discover

2. Search:

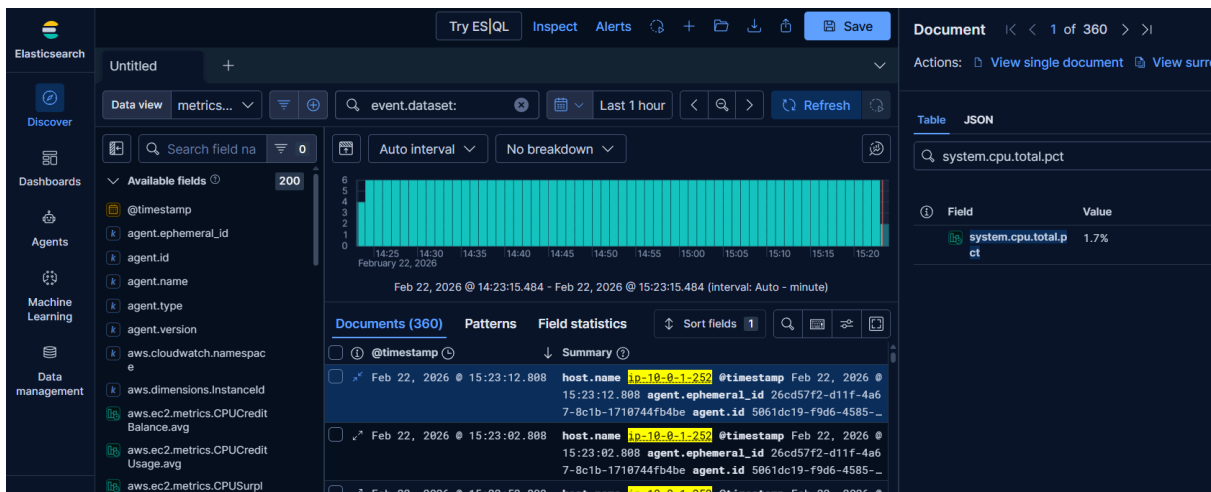
`event.dataset: "system.cpu" AND host.name: *10-0-1*`

Or go to Kibana → Dashboard and search for "System" — there are pre-built dashboards showing CPU, memory, and disk per host. In Kibana Discover, Make sure you switch from logs-* to metric-* when monitoring metrics. This is located just below the name of the analysis tab or *Data View*.

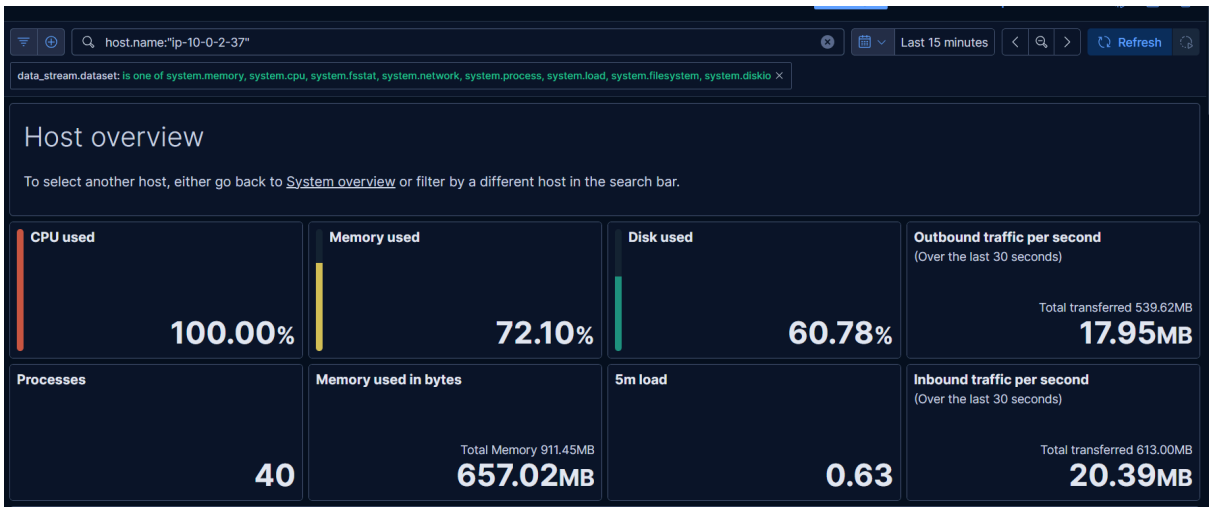
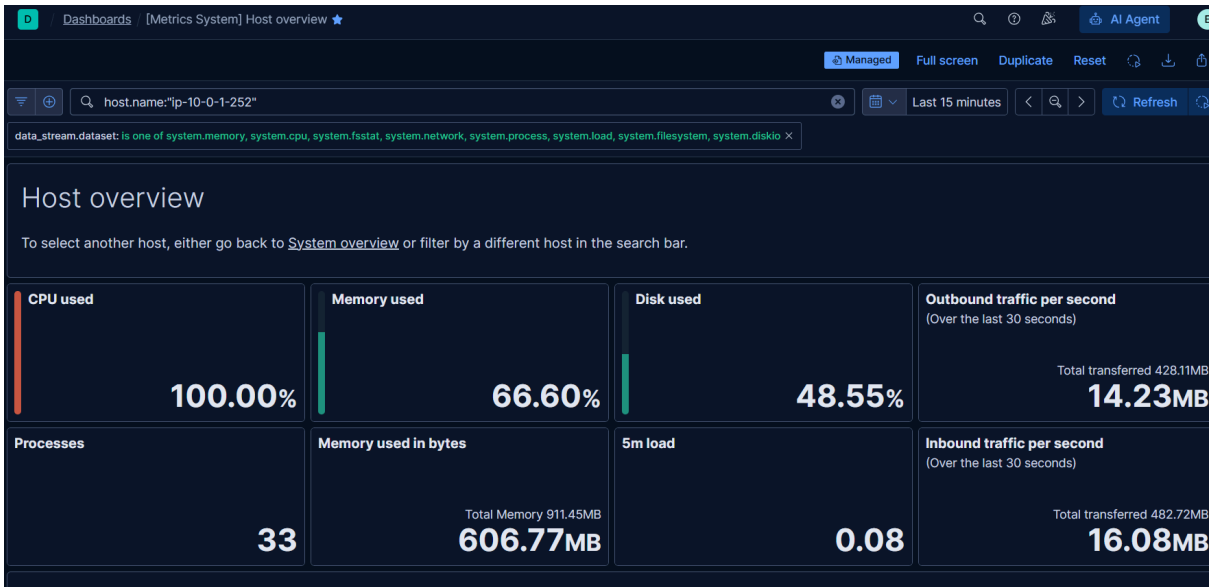
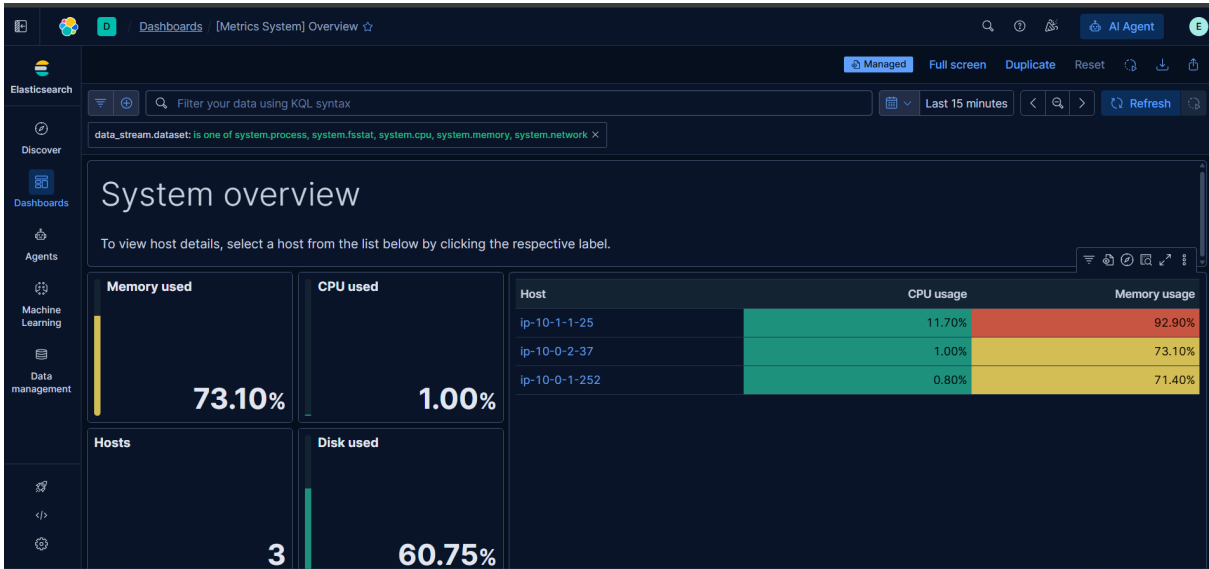


Look for a spike in **system.cpu.total.pct** hitting close to 1.0 (which means 100%).

(Using Kibana Discover)



(Using Dashboard – [Metrics System] Host overview)



Category C Summary Checklist

Element	Test	Success Condition
C.1 S3 Enumeration	Loop <code>aws s3 ls</code> 10 times	ListBucket events in <code>aws.s3access</code>
C.2 VPC Flow REJECT	curl blocked ports on Backend	REJECT entries in <code>aws.vpcflow</code>
C.3 Metric Accuracy	stress --cpu on Frontend/Backend	CPU spike visible in <code>system.cpu</code>

Category D: Adversarial Simulation

This is the most exciting category — we're simulating real attacks and verifying your SOC catches them. All tests are safe and contained within your own infrastructure.

D.1: Layer 7 DDoS Simulation

Description: Flood your Frontend EC2 with HTTP requests simulating a DDoS attack and verify Kibana captures the traffic spike.

Purpose: Validate that your SOC can detect abnormal traffic volume against your public-facing infrastructure.

Success Condition: Massive spike in network connections visible in Kibana, VPC Flow Logs show traffic surge from a single source.

How to Test:

First, install the load testing tool on your SOC instance:

```
sudo apt-get install apache2-utils -y
```

Then hammer your Frontend:

```
# 10000 requests, 100 concurrent
```

```
ab -n 10000 -c 100 http://<FRONTEND_PRIVATE_IP>/
```

Simultaneously, from your local laptop using PowerShell:

```
for ($i = 0; $i -lt 500; $i++) {
```

```
    Invoke-WebRequest -Uri "http://<FRONTEND_PUBLIC_IP>" -UseBasicParsing
```

```
}
```

✓ Check in Kibana:

VPC Flow Logs spike:

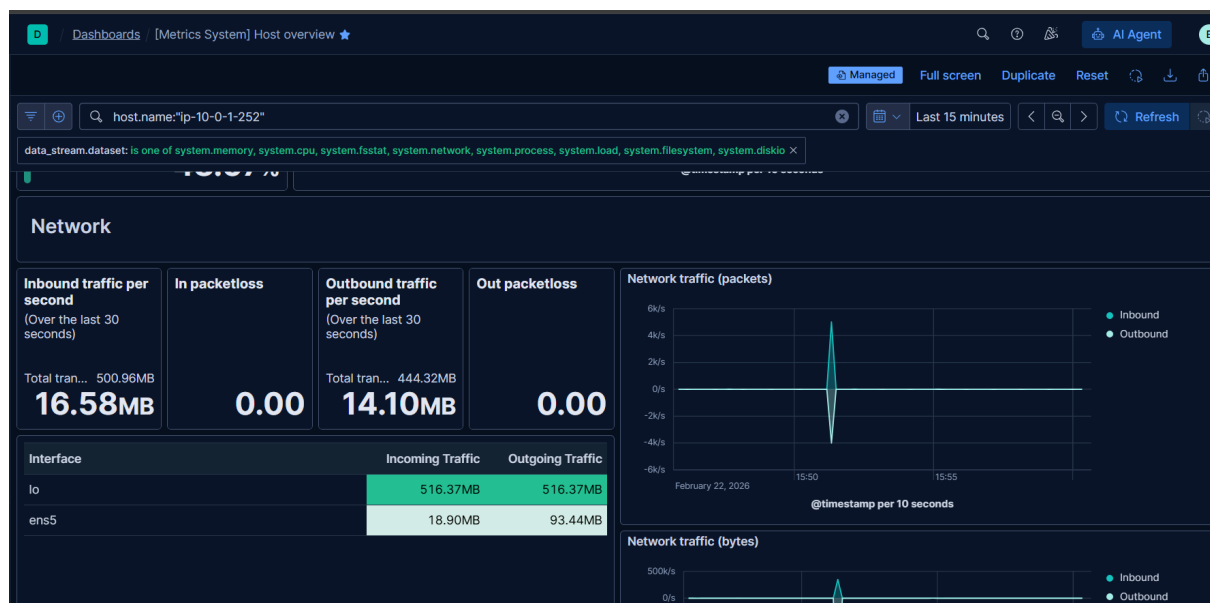
```
event.dataset: "aws.vpcflow" AND destination.ip: <FRONTEND_PRIVATE_IP>
```

Network metrics on Frontend:

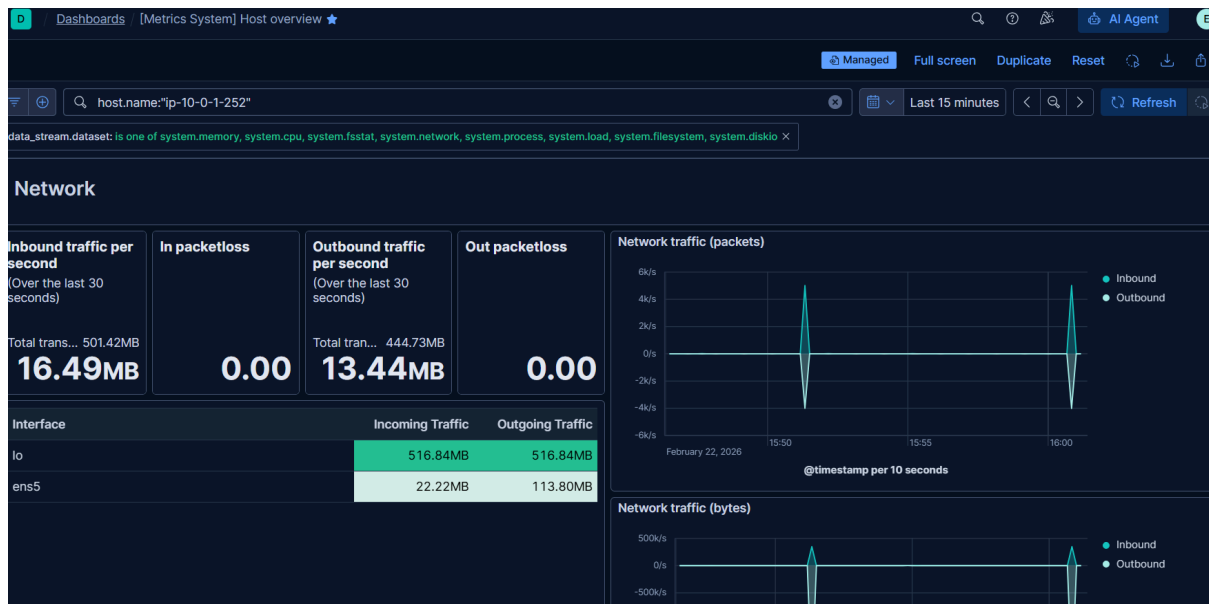
```
event.dataset: "system.network" AND host.name: *10-0-1*
```

Check the System Host Overview dashboard — you should see a massive spike in network traffic during the test.

Before:



After:



You can see the traffic jumps at the end there.

D.2: Malicious Payload Upload Test

Description: Attempt to upload a fake malicious file to your S3 bucket and verify it's captured in S3 access logs.

Purpose: Detect unauthorized or suspicious file uploads to company storage.

Success Condition: PutObject event appears in Kibana with the filename and requester identity.

How to Test:

First, temporarily add PutObject and DeleteObject to the RestrictedDataAccess. You can delete it later because the SOC platform shouldn't be able to add or modify any resources. Monitoring and alerting only.

```

{
  "Sid": "RestrictedDataAccess",
  "Effect": "Allow",
  "Action": [
    "s3:GetObject",
    "s3:ListBucket",
    "s3:GetBucketLocation",
    "s3:PutObject",
    "s3:DeleteObject"
  ],
  "Resource": [
    "arn:aws:s3::aws-cloudtrail-logs-mj",
    "arn:aws:s3::aws-cloudtrail-logs-mj/*",
    "arn:aws:s3::company-storage-network-project",
    "arn:aws:s3::company-storage-network-project/*",
    "arn:aws:s3::company-storage-access-logs",
    "arn:aws:s3::company-storage-access-logs/*"
  ]
}

```

Next, from your SOC instance:

Create a fake malicious file

```
echo "FAKE_MALWARE_TEST_DO_NOT_EXECUTE" > malware-test.txt
```

Try uploading it to the company bucket

```
aws s3 cp malware-test.txt s3://company-storage-network-project/malware-test.txt --region us-east-1
```

Clean it up immediately after

```
aws s3 rm s3://company-storage-network-project/malware-test.txt --region us-east-1
```

Clean local file

```
rm malware-test.txt
```

✅ Check in Kibana:

Wait 15 minutes, then search:

```
data_stream.dataset: "aws.s3access" AND aws.s3access.operation: "PUT*"
```

Look for:

- aws.s3access.operation — PutObject
- aws.s3access.key — malware-test.txt

- aws.s3access.requester — your SOC role ARN
- aws.s3access.http_status — 200

D.3: SQL Injection Probing

Description: Send SQLi payloads through your Frontend proxy targeting the Backend/RDS and verify they appear in logs.

Purpose: SQL injection is one of the most common attacks against web applications with databases. Your SOC should detect the attempt.

Success Condition: SQLi patterns appear in Frontend access logs and RDS general logs in Kibana.

How to Test:

From your local laptop, send classic SQLi payloads to your Frontend:

Basic SQLi attempts

```
curl "http://<FRONTEND_PUBLIC_IP>/login?user=admin'--"
```

```
curl "http://<FRONTEND_PUBLIC_IP>/login?user=1' OR '1'='1"
```

```
curl "http://<FRONTEND_PUBLIC_IP>/login?user=1; DROP TABLE users--"
```

```
curl "http://<FRONTEND_PUBLIC_IP>/search?q=' UNION SELECT * FROM users--"
```

[WARNING]: The code below is a real SQL attack CLI. This CLI will show

```
curl "http://<FRONTEND_PUBLIC_IP>/api/files?search=' UNION SELECT 1,2,3,4--"
```

✅ Check in Kibana:

Check Frontend access logs:

```
event.dataset: "system.syslog" AND message: *UNION*
```

```
event.dataset: "system.syslog" AND message: *DROP TABLE*
```

Check RDS general logs if the payload reached the DB:

aws.cloudwatch.log_group: *general* AND message: *UNION*

```
PS C:\Users\User> curl "http://44.198.171.210/api/files?search=1'; DROP TABLE files--"

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is
parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): A

StatusCode      : 200
StatusDescription : OK
Content         : []
RawContent      : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Content-Length: 2
                  Content-Type: application/json; charset=utf-8
                  Date: Sun, 22 Feb 2026 09:27:45 GMT
                  ETag: W/"2-l9Fw4VU07kr8CvBlt4zaMCqXZ0w"
                  Server: nginx/1...
Forms           : {}
Headers         : {[Connection, keep-alive], [Content-Length, 2], [Content-Type, application/json; charset=utf-8],
                  [Date, Sun, 22 Feb 2026 09:27:45 GMT]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 2
```

event.dataset: Last 15 minutes Refresh

Search field names or values 0

Popular fields 1 Available fields 170

@timestamp actor.entity.id agent.ephemeral.id agent.id agent.name agent.type agent.version aws.cloudtrail.additional_eventdata aws.cloudtrail.api_version aws.cloudtrail.console_log_in_additional_eventdata.log_in_to aws.cloudtrail.console_log_in_additional_eventdata.mfa

1 1 1 0 0 0 16:15 16:20 16:25 February 22, 2026 Feb 22, 2026 @ 16:13:37.792 - Feb 22, 2026 @ 16:28:37.792 (Interval: Auto - 30 seconds)

Documents (1) Patterns Field statistics Sort fields 1

Feb 22, 2026 @ 16:26:48.861 message 2026-02-22T09:26:48.861Z - GET /api/files?search=1'; DROP TABLE files-- @timestamp Feb 22, 2026 @ 16:26:48.861

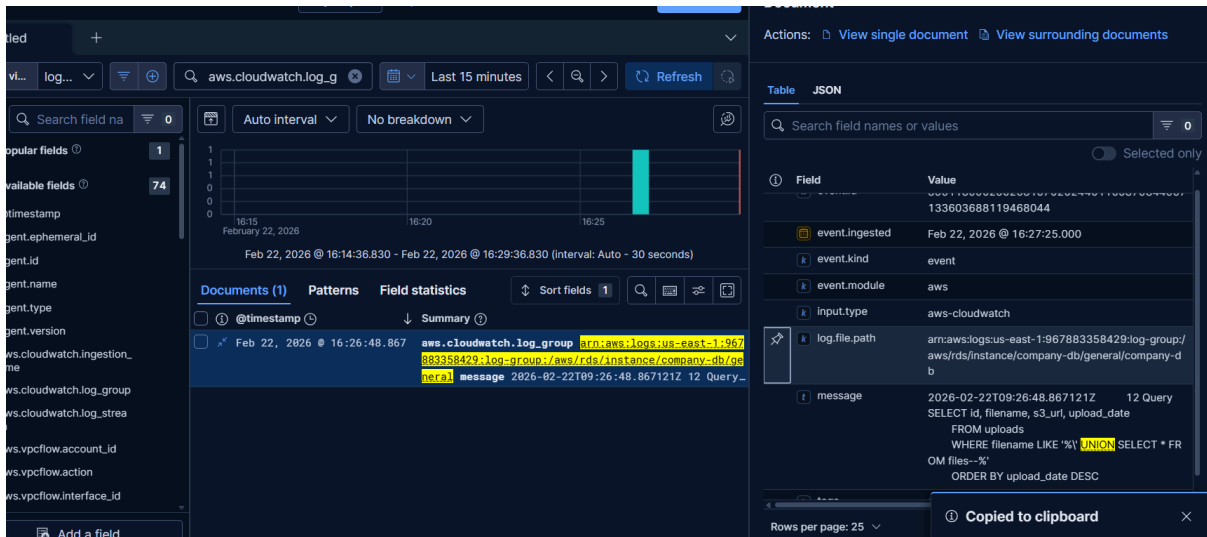
Actions: View single document View surrounding documents

Table JSON

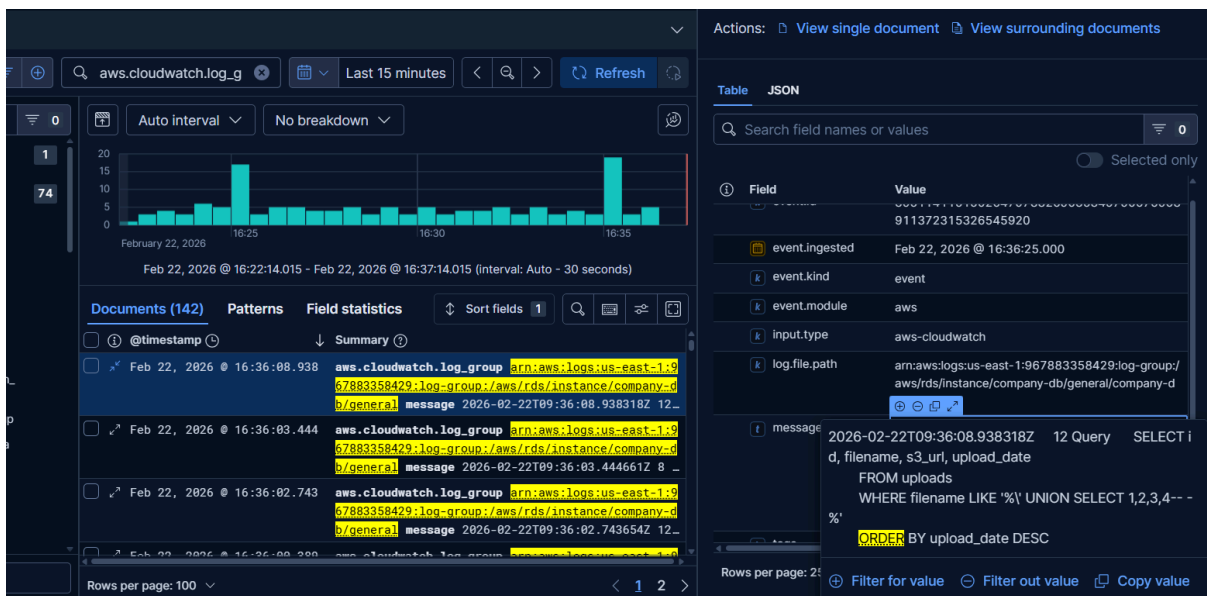
Search field names or values 0 Selected only

Field	Value
host.os.name	Ubuntu
host.os.platform	ubuntu
host.os.type	linux
host.os.version	24.04.3 LTS (Noble Numbat)
input.type	log
log.file.path	/var/log/syslog
log.offset	
message	2026-02-22T09:26:48.861Z - GET /api/files?search=1'; DROP TABLE files--
process.name	node
process.pid	5,629

Rows per page: 25 < 1 2 >



(From the real SQL attack CLI)



From PowerShell:

```
PS C:\Users\User> curl "http://44.198.171.210/api/files?search=' UNION SELECT 1,2,3,4--'"

StatusCode      : 200
StatusDescription : OK
Content         : {}
RawContent      : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Content-Length: 2
                  Content-Type: application/json; charset=utf-8
                  Date: Sun, 22 Feb 2026 09:35:03 GMT
                  ETag: W/"2-L9Fw4VU07kr8CvBlt4zaMcqXZ0w"
                  Server: nginx/1...

Forms           : {}
Headers         : {[Connection, keep-alive], [Content-Length, 2], [Content-Type, application/json; charset=utf-8],
                  [Date, Sun, 22 Feb 2026 09:35:03 GMT]...}

Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 2
```


As you can see, the Content row is empty. It means that the app is protected from basic SQL attacks.

 Category D Summary Checklist

Element	Test	Success Condition
D.1 DDoS Simulation	10000 HTTP requests via ab	Traffic spike in VPC Flow Logs and system.network
D.2 Malicious Upload	Upload fake file to S3	PutObject event in aws.s3access
D.3 SQL Injection	SQLi payloads to Frontend	Patterns visible in syslog and RDS general logs

END